Deep Network Development

Autumn 2026, Practice #3

Viktor Varga
Department of Artificial Intelligence, ELTE IK

NumPy - Introduction



NumPy - Introduction



NumPy - Introduction

The classic approach

```
my_list = [1, 2, 3, -42, 99]

for i in range(len(my_list)):
    my_list[i] += 1

print(my_list)
```

NumPy - Introduction

The classic approach

```
my_list = [1, 2, 3, -42, 99]

for i in range(len(my_list)):
    my_list[i] += 1

print(my_list)

>>> [2, 3, 4, -41, 100]
```

NumPy - Introduction

The classic approach

```
my_list = [1, 2, 3, -42, 99]

for i in range(len(my_list)):
    my_list[i] += 1

print(my_list)

>>> [2, 3, 4, -41, 100]
```

Array programming

NumPy - Introduction

```
import numpy as np
```

The classic approach

```
my_list = [1, 2, 3, -42, 99]

for i in range(len(my_list)):
    my_list[i] += 1

print(my_list)

>>> [2, 3, 4, -41, 100]
```

Array programming

NumPy - Introduction

The classic approach

```
my_list = [1, 2, 3, -42, 99]

for i in range(len(my_list)):
    my_list[i] += 1

print(my_list)

>>> [2, 3, 4, -41, 100]
```

```
import numpy as np
```

Array programming

```
my_array = np.array([1, 2, 3, -42, 99])

my_array += 1

print(my_array)
```

NumPy - Introduction

The classic approach

```
my_list = [1, 2, 3, -42, 99]

for i in range(len(my_list)):
    my_list[i] += 1

print(my_list)

>>> [2, 3, 4, -41, 100]
```

```
import numpy as np
```

Array programming

```
my_array = np.array([1, 2, 3, -42, 99])

my_array += 1

print(my_array)

>>> [  2   3   4 -41 100]
```

NumPy - Introduction

The classic approach

```
my_list = [1, 2, 3, -42, 99]

for i in range(len(my_list)):
    my_list[i] += 1
```

Scalar operation: scalar + scalar

```
print(my_list)

>>> [2, 3, 4, -41, 100]
```

Array programming

```
import numpy as np

my_array = np.array([1, 2, 3, -42, 99])

my_array += 1

print(my_array)

>>> [  2   3   4 -41 100]
```

NumPy - Introduction

The classic approach

```
my_list = [1, 2, 3, -42, 99]

for i in range(len(my_list)):
    my_list[i] += 1
```

Scalar operation: scalar + scalar

```
print(my_list)

>>> [2, 3, 4, -41, 100]
```

Array programming

```
import numpy as np

my_array = np.array([1, 2, 3, -42, 99])

my_array += 1
```

Vectorized operation: array + scalar

```
print(my_array)

>>> [  2   3   4 -41 100]
```

NumPy - Introduction



Example: Several triplets of numbers are given. Specify which ones can be used to form triangles, and then specify the area and perimeter of these triangles.

```
def triangle_area_perimeter(edges):
  valid_tr_mask = (edges[:,0] + edges[:,1] > edges[:,2]) & (edges[:,0] + edges[:,2] > edges[:,1]) &\
                  (edges[:,1] + edges[:,2] > edges[:,0]) & np.all(edges >= 0., axis=1)
  valid_triangle_idxs = np.where(valid_tr_mask)[0]
  edges = edges[valid_triangle_idxs]
  perimeter = np.sum(edges, axis=1)
  semi_per = perimeter*.5
  area = np.sqrt(semi_per*(semi_per-edges[:,0])*(semi_per-edges[:,1])*(semi_per-edges[:,2]))
  area_perimeter = np.stack([area, perimeter], axis=1)
  return valid_triangle_idxs, area_perimeter
```

NumPy - Introduction

Example: Several triplets of numbers are given. Specify which ones can be used to form triangles, and then specify the area and perimeter of these triangles.

Vectorized operation: array + array

```
def triangle_area_perimeter(edges):
  valid_tr_mask = (edges[:,0] + edges[:,1] > edges[:,2]) & (edges[:,0] + edges[:,2] > edges[:,1]) &\
                  (edges[:,1] + edges[:,2] > edges[:,0]) & np.all(edges >= 0., axis=1)
  valid_triangle_idxs = np.where(valid_tr_mask)[0]
  edges = edges[valid_triangle_idxs]
  perimeter = np.sum(edges, axis=1)
  semi_per = perimeter*.5
  area = np.sqrt(semi_per*(semi_per-edges[:,0])*(semi_per-edges[:,1])*(semi_per-edges[:,2]))
  area_perimeter = np.stack([area, perimeter], axis=1)
  return valid_triangle_idxs, area_perimeter
```

NumPy - Introduction

Example: Several triplets of numbers are given. Specify which ones can be used to form triangles, and then specify the area and perimeter of these triangles.

Vectorized operation: array + array

N dimensional arrays, indexing different axes in different ways

```
def triangle_area_perimeter(edges):
  valid_tr_mask = (edges[:,0] + edges[:,1] > edges[:,2]) & (edges[:,0] + edges[:,2] > edges[:,1]) &\
                  (edges[:,1] + edges[:,2] > edges[:,0]) & np.all(edges >= 0., axis=1)
  valid_triangle_idxs = np.where(valid_tr_mask)[0]
  edges = edges[valid_triangle_idxs]
  perimeter = np.sum(edges, axis=1)
  semi_per = perimeter*.5
  area = np.sqrt(semi_per*(semi_per-edges[:,0])*(semi_per-edges[:,1])*(semi_per-edges[:,2]))
  area_perimeter = np.stack([area, perimeter], axis=1)
  return valid_triangle_idxs, area_perimeter
```

NumPy - Introduction

Example: Several triplets of numbers are given. Specify which ones can be used to form triangles, and then specify the area and perimeter of these triangles.

Vectorized operation: array + array

N dimensional arrays, indexing different axes in different ways

```
def triangle_area_perimeter(edges):
  valid_tr_mask = (edges[:,0] + edges[:,1] > edges[:,2]) & (edges[:,0] + edges[:,2] > edges[:,1]) &\
                  (edges[:,1] + edges[:,2] > edges[:,0]) & np.all(edges >= 0., axis=1)
  valid_triangle_idxs = np.where(valid_tr_mask)[0]
  edges = edges[valid_triangle_idxs]
  perimeter = np.sum(edges, axis=1)
  semi_per = perimeter*.5
  area = np.sqrt(semi_per*(semi_per-edges[:,0])*(semi_per-edges[:,1])*(semi_per-edges[:,2]))
  area_perimeter = np.stack([area, perimeter], axis=1)
  return valid_triangle_idxs, area_perimeter
```

Using an array of indices at once

NumPy - Introduction

Example: Several triplets of numbers are given. Specify which ones can be used to form triangles, and then specify the area and perimeter of these triangles.

Vectorized operation: array + array

N dimensional arrays, indexing different axes in different ways

```
def triangle_area_perimeter(edges):
  valid_tr_mask = (edges[:,0] + edges[:,1] > edges[:,2]) & (edges[:,0] + edges[:,2] > edges[:,1]) &\
                  (edges[:,1] + edges[:,2] > edges[:,0]) & np.all(edges >= 0., axis=1)
  valid_triangle_idxs = np.where(valid_tr_mask)[0]
  edges = edges[valid_triangle_idxs]
  perimeter = np.sum(edges, axis=1)
  semi_per = perimeter*.5
  area = np.sqrt(semi_per*(semi_per-edges[:,0])*(semi_per-edges[:,1])*(semi_per-edges[:,2]))
  area_perimeter = np.stack([area, perimeter], axis=1)
  return valid_triangle_idxs, area_perimeter
```

Using an array of indices at once

Executing operations independently on multiple slices of an array

NumPy - Introduction

The classic approach

```
my_list = [1, 2, 3, -42, 99]

for i in range(len(my_list)):
    my_list[i] += 1
```

Scalar operation: scalar + scalar

```
print(my_list)

>>> [2, 3, 4, -41, 100]
```

Array programming

```
import numpy as np

my_array = np.array([1, 2, 3, -42, 99])

my_array += 1
```

Vectorized operation: array + scalar

```
print(my_array)

>>> [  2   3   4 -41 100]
```

Image source: https://milai.tech/products/PixelsWorld/docs/contents/GLSL/GLSLCode.html

Hardware options

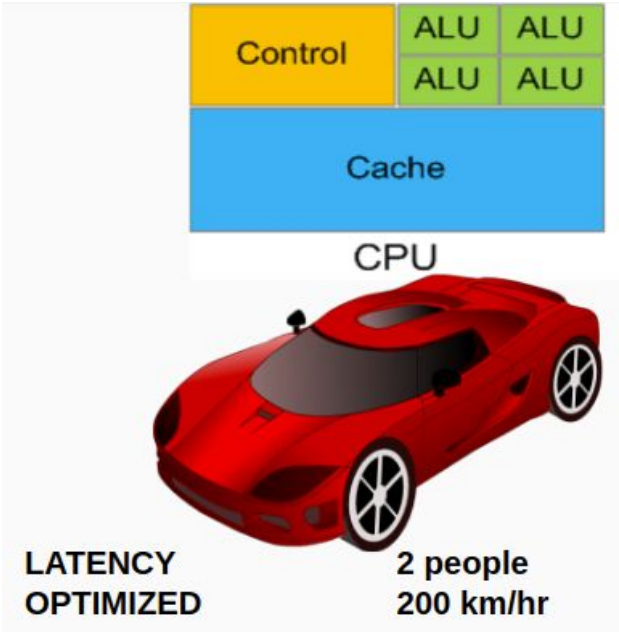


Image source: https://milai.tech/products/PixelsWorld/docs/contents/GLSL/GLSLCode.html

Hardware options

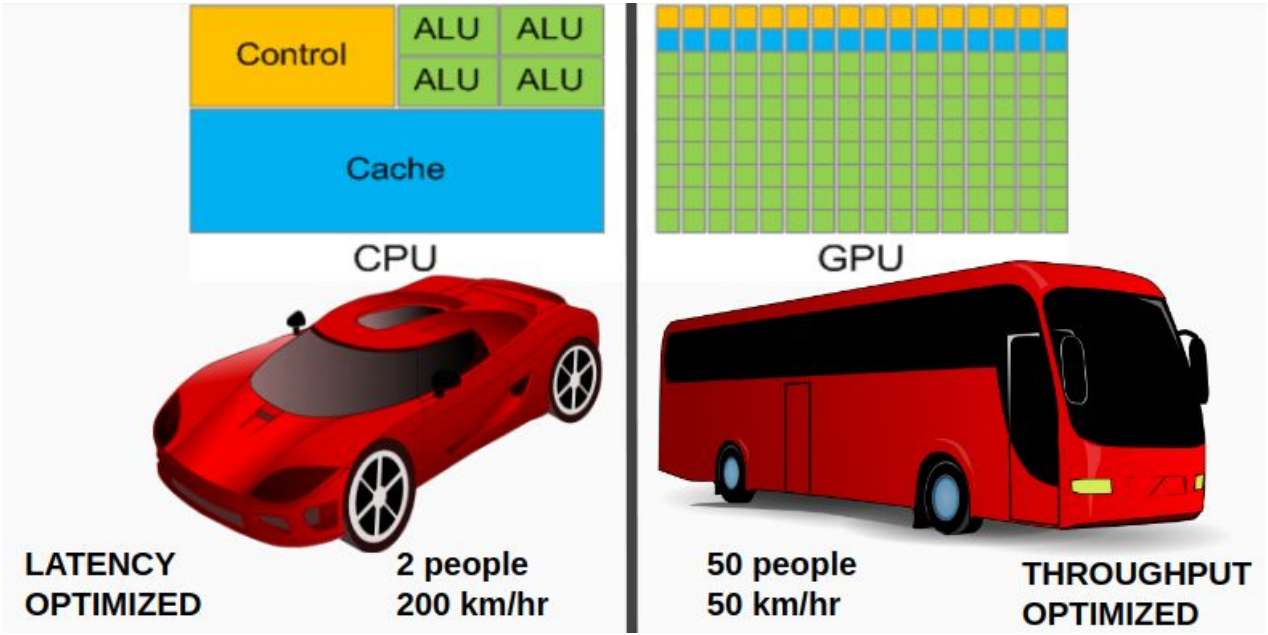
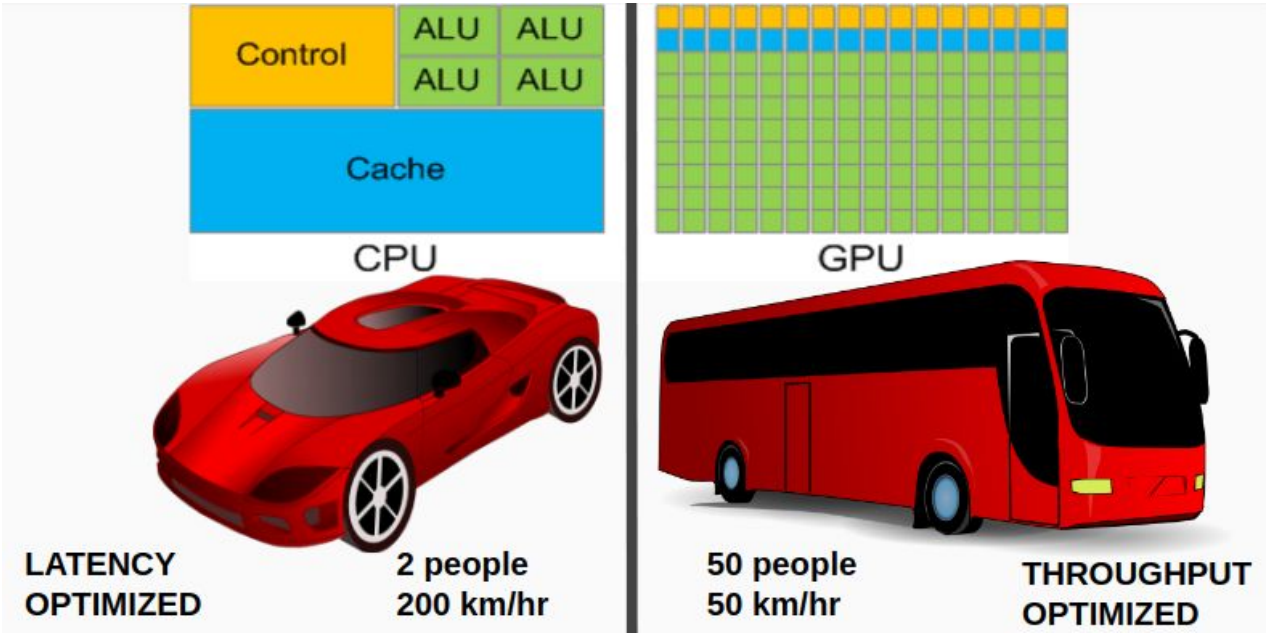


Image source: https://milai.tech/products/PixelsWorld/docs/contents/GLSL/GLSLCode.html

Hardware options



Vector processors / SIMD architectures

NumPy - Introduction

The classic approach

```
my_list = [1, 2, 3, -42, 99]

for i in range(len(my_list)):
    my_list[i] += 1
```

Scalar operation: scalar + scalar

```
print(my_list)

>>> [2, 3, 4, -41, 100]
```

Array programming

```
import numpy as np

my_array = np.array([1, 2, 3, -42, 99])

my_array += 1
```

Vectorized operation: array + scalar

```
print(my_array)

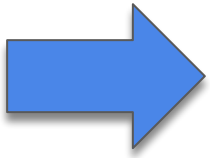
>>> [  2   3   4 -41 100]
```

NumPy - Introduction



NumPy - Introduction







Python - Features

- **Interpreters vs. compilers**

Python - Features

- Interpreters vs. compilers
- **Type checking at runtime**

Python - Features

- Interpreters vs. compilers
- **Type checking at runtime**

```
a = "abc" + 2
```

```
>>> TypeError: can only concatenate str (not "int") to str
```

Python - Features

- Interpreters vs. compilers
- Type checking at runtime
- **Automatic memory management**

Image source: https://www.nickmccullum.com/python-pointers/

Python - Features

- Interpreters vs. compilers
- Type checking at runtime
- **Automatic memory management**

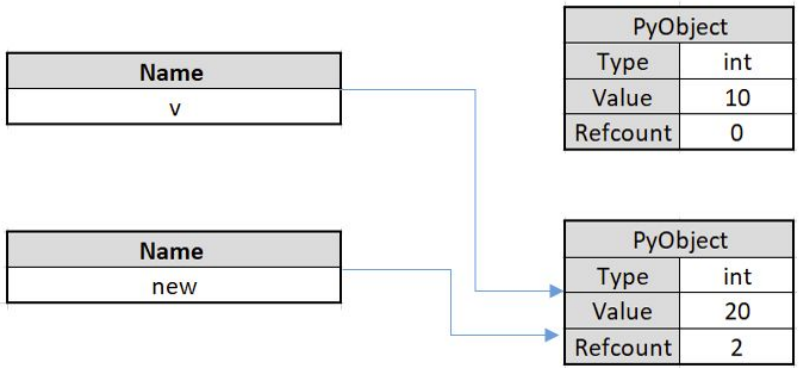
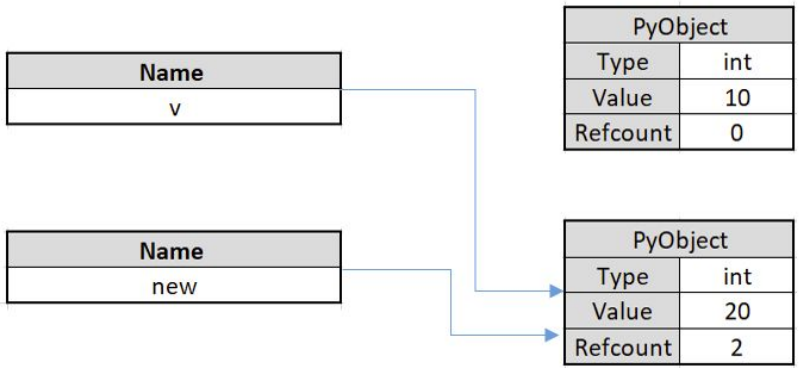


Image source: https://www.nickmccullum.com/python-pointers/

Python - Features

- Interpreters vs. compilers
- Type checking at runtime
- Automatic memory management
- **No native types, everything is an object**

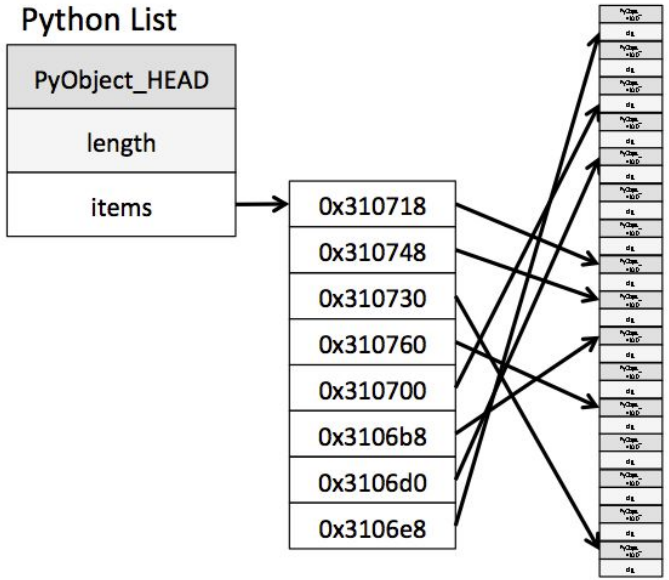


Python - Features

- Interpreters vs. compilers
- Type checking at runtime
- Automatic memory management
- No native types, everything is an object
- **Built-in data structures favor convenience over efficiency**
  - **Mixed type elements:** Storing object references instead of values

Python - Features

- Interpreters vs. compilers
- Type checking at runtime
- Automatic memory management
- No native types, everything is an object
- **Built-in data structures favor convenience over efficiency**
  - **Mixed type elements:** Storing object references instead of values



Python - Features

- Interpreters vs. compilers
- Type checking at runtime
- Automatic memory management
- No native types, everything is an object
- Built-in data structures favor convenience over efficiency
  - Mixed type elements: Storing object references instead of values
- **High portability**

Python - Features

- Interpreters vs. compilers
- Type checking at runtime
- Automatic memory management
- No native types, everything is an object
- Built-in data structures favor convenience over efficiency
  - Mixed type elements: Storing object references instead of values
- High portability
- **Simplicity**

Python - Features



- Interpreters vs. compilers
- Type checking at runtime
- Automatic memory management
- No native types, everything is an object
- Built-in data structures favor convenience over efficiency
  - Mixed type elements: Storing object references instead of values
- High portability
- Simplicity
- **Large ecosystem of tools and extensions**

Image source: https://www.geeksforgeeks.org/python/python-lists-vs-numpy-arrays/

NumPy - The key to efficiency

- **numpy.ndarray:** A container storing native values, not references

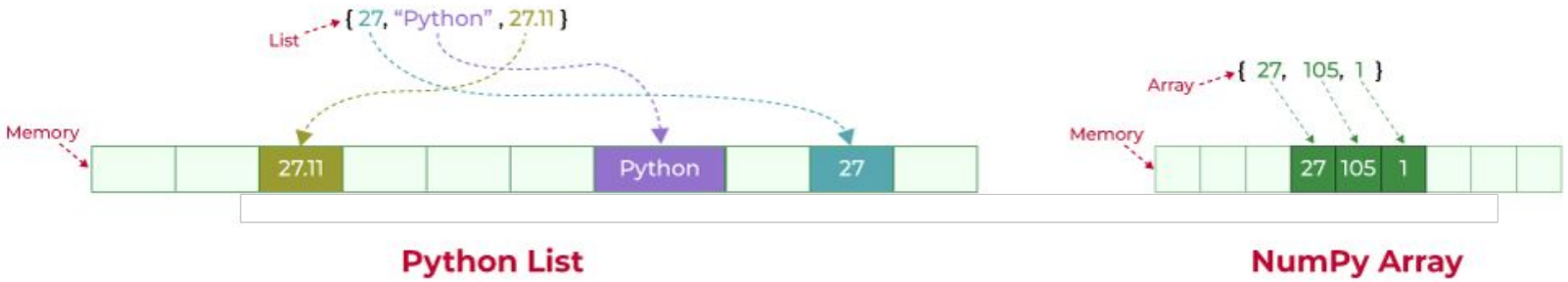
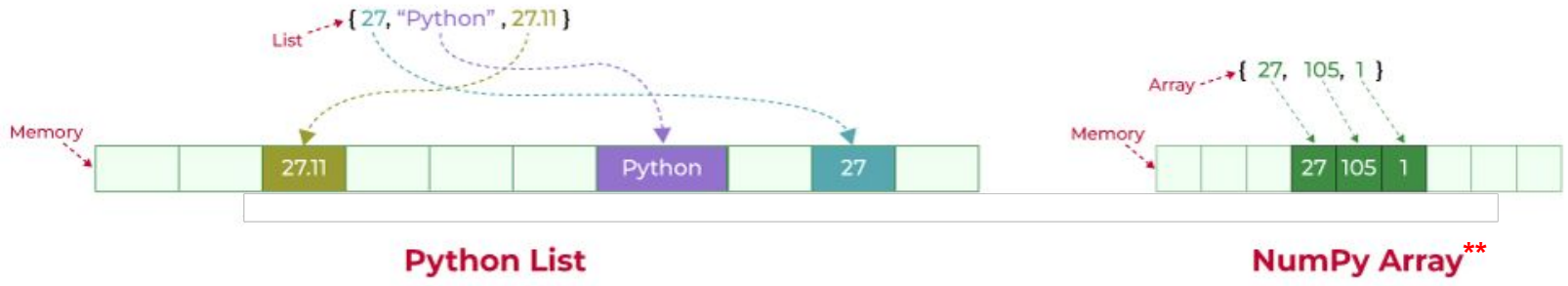


Image source: https://www.geeksforgeeks.org/python/python-lists-vs-numpy-arrays/

NumPy - The key to efficiency

- **numpy.ndarray:** A container storing native values, not references*



* Except for `dtype=object`

** Only true for base arrays, not necessarily for views.

NumPy - The key to efficiency

- numpy.ndarray: A container storing native values
- **Efficient implementation of NumPy functions / operations**

NumPy - The key to efficiency

- numpy.ndarray: A container storing native values
- Efficient implementation of NumPy functions / operations

Not to forget: When using NumPy, we are still within the framework of Python.

There is an overhead on each single Python expression / statement!

NumPy - The key to efficiency

- numpy.ndarray: A container storing native values
- Efficient implementation of NumPy functions / operations

Not to forget: When using NumPy, we are still within the framework of Python.

There is an overhead on each single Python expression / statement!

- As few (NumPy) operations as possible with as large arrays as possible
  → **Efficient**
- Many operations applied to small amounts of data at a time
  → **Inefficient**

Image source: https://www.geeksforgeeks.org/python/python-lists-vs-numpy-arrays/

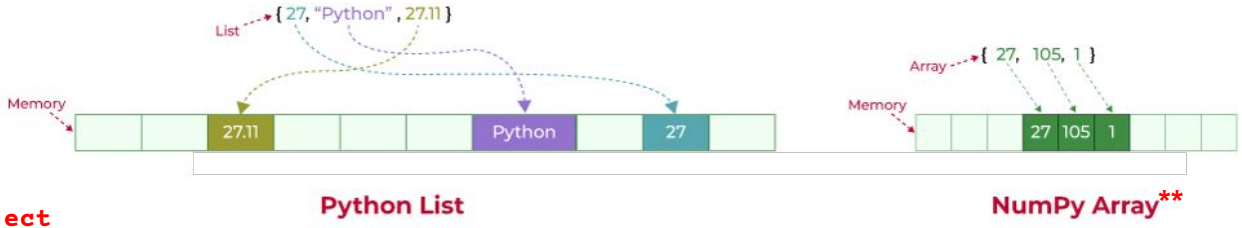
Python lists vs. NumPy arrays

Python lists

- Can have **mixed type** elements
  → **Not efficient** (references)

NumPy arrays

- Elements have a **fixed datatype***
  → **Efficient** (native storage)



* Except for `dtype=object`

** Only true for base arrays, not necessarily for views.

Image source: https://www.geeksforgeeks.org/python/python-lists-vs-numpy-arrays/

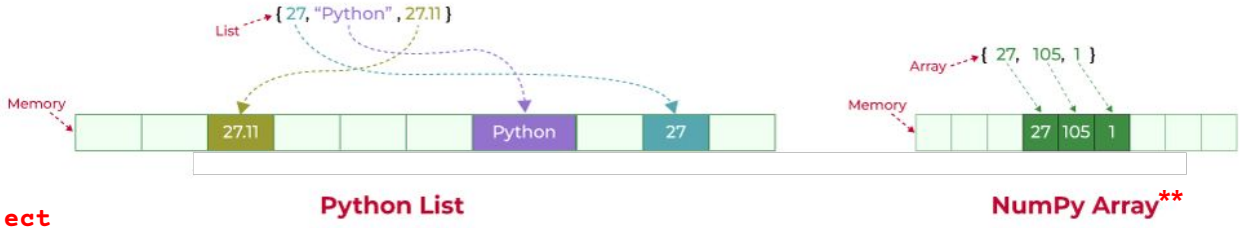
Python lists vs. NumPy arrays

Python lists

- Can have **mixed type** elements
  → **Not efficient** (references)
- Has some free space at the end
  → **list.append is O(1)**

NumPy arrays

- Elements have a **fixed datatype***
  → **Efficient** (native storage)
- Completely fills reserved memory**
  → **ndarray.append is O(n)**



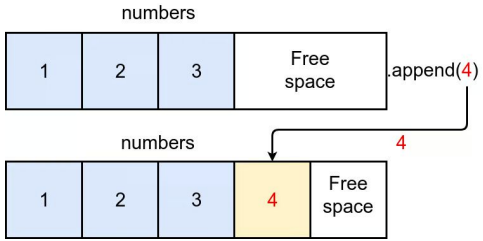
* Except for `dtype=object`
** Only true for base arrays, not necessarily for views.

Image sources: https://www.geeksforgeeks.org/python/python-lists-vs-numpy-arrays/, https://realpython.com/python-append/

Python lists vs. NumPy arrays

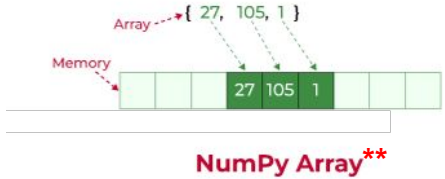
Python lists

- Can have **mixed type** elements
  → **Not efficient** (references)
- Has some free space at the end
  → **list.append is O(1)**



NumPy arrays

- Elements have a **fixed datatype***
  → **Efficient** (native storage)
- Completely fills reserved memory**
  → **ndarray.append is O(n)**



* Except for `dtype=object`
** Only true for base arrays, not necessarily for views.

Image source: https://www.geeksforgeeks.org/python/python-lists-vs-numpy-arrays/

Python lists vs. NumPy arrays

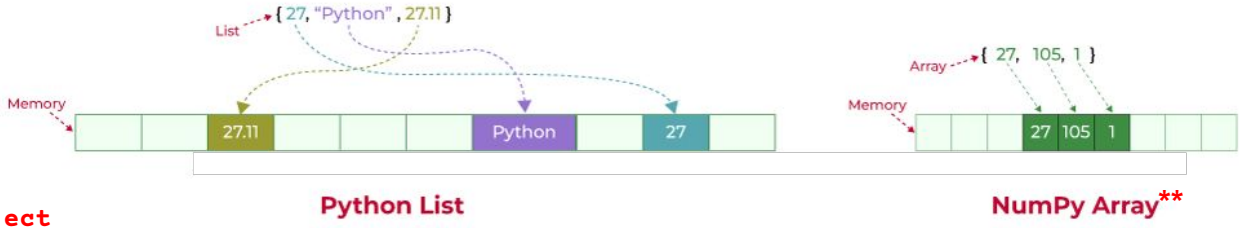
Python lists

- Can have **mixed type** elements
  → **Not efficient** (references)
- Has some free space at the end
  → **list.append is O(1)**

NumPy arrays

- Elements have a **fixed datatype***
  → **Efficient** (native storage)
- Completely fills reserved memory**
  → **ndarray.append is O(n)**

Do not append elements to an array iteratively!!!
Use a list for this purpose, or create a larger array.



* Except for `dtype=object`
** Only true for base arrays, not necessarily for views.

Image source: https://www.geeksforgeeks.org/python/python-lists-vs-numpy-arrays/

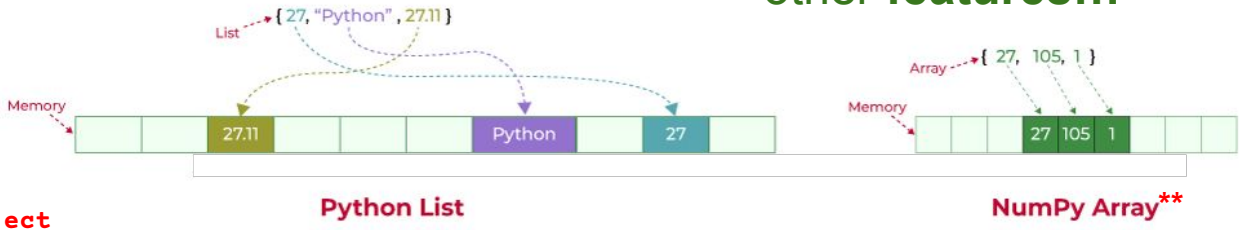
Python lists vs. NumPy arrays

Python lists

- Can have **mixed type** elements
  → **Not efficient** (references)
- Has some free space at the end
  → **list.append is O(1)**

NumPy arrays

- Elements have a **fixed datatype***
  → **Efficient** (native storage)
- Completely fills reserved memory**
  → **ndarray.append is O(n)**
- Can be **multi-dimensional**
- Much more **powerful** indexing and other **features…**



* Except for `dtype=object`
** Only true for base arrays, not necessarily for views.

NumPy - The ndarray type

The ndarray type – Main attributes:

- `ndarray.size`
- `ndarray.ndim`
- `ndarray.shape`
- `ndarray.dtype`
- `ndarray.itemsize`
- `ndarray.strides`
- `ndarray.base`

NumPy - The ndarray type

The ndarray type – Main attributes:

```
a = np.array([0., 10., 20., 30., 40.], dtype=np.float32)
```

- `ndarray.size` `5`
- `ndarray.ndim` `1`
- `ndarray.shape` `(5,)`
- `ndarray.dtype` `float32`
- `ndarray.itemsize` `4`
- `ndarray.strides`
- `ndarray.base`

NumPy - The ndarray type

The ndarray type – Main attributes:

```
a = np.array([0., 10., 20., 30., 40.], dtype=np.float32)
```

- `ndarray.size` → `5`
- `ndarray.ndim` → `1`
- `ndarray.shape` → `(5,)` ← A tuple
- `ndarray.dtype` → `float32`
- `ndarray.itemsize` → `4`
- `ndarray.strides`
- `ndarray.base`

If array is 1D (`ndarray.ndim == 1`) →
`ndarray.shape == (ndarray.size,)`

NumPy - Frequently used data types (dtype)

Integer dtypes:

```
np.int8
np.int16
np.int32
np.int64

np.uint8
np.uint16
np.uint32
np.uint64
```

Floating point dtypes

```
np.float16
np.float32
np.float64
```

Logical dtype

```
np.bool_
```

Array of pointers

```
dtype=object
```

NumPy - Frequently used data types (dtype)

Integer dtypes:

```
np.int8
np.int16
np.int32
np.int64

np.uint8
np.uint16
np.uint32
np.uint64

42
```

Floating point dtypes

```
np.float16
np.float32
np.float64

-42.5
```

Logical dtype

```
np.bool_

True
```

Array of pointers

```
dtype=object

{[[], (,)], 2, “abc”}
```

NumPy - Frequently used data types (dtype)

Integer dtypes:

E.g., `np.iinfo(np.int8).max`

`np.int8`	`-128 .. 127`
`np.int16`	`-32768 .. 32767`
`np.int32`	`-2147483648 .. 2147483647`
`np.int64`	`-9223372036854775808 .. 9223372036854775807`
`np.uint8`	`0 .. 255`
`np.uint16`	`0 .. 65536`
`np.uint32`	`0 .. 4294967295`
`np.uint64`	`0 .. 18446744073709551615`

NumPy - Frequently used data types (dtype)

Integer dtypes:

```
np.int8
np.int16
np.int32
np.int64

np.uint8
np.uint16
np.uint32
np.uint64
```

Floating point dtypes

`np.float16`	`-65504 .. 65504`
`np.float32`	`≈-3.4e+38 .. ≈3.4e+38`
`np.float64`	`≈-1.8e+308 .. ≈1.8e+308`

Logical dtype

`np.bool_`	`in [False, True]`

Array of pointers

`dtype=object`

E.g., `np.finfo(np.float64).min`

NumPy - Frequently used data types (dtype)

Integer dtypes:

```
np.int8
np.int16
np.int32
np.int64

np.uint8
np.uint16
np.uint32
np.uint64
```

Floating point dtypes

```
np.float16
np.float32
np.float64
```

Logical dtype

```
np.bool_
```

Array of pointers

```
dtype=object
```

Any element can be any Python object.

Inefficient to use!

Python type vs. NumPy ndarray.dtype

Avoid mixing up **Python types** and **ndarray.dtype!**

- **Python type**
  - All Python objects have a type
  - The type of NumPy arrays is **`ndarray`**

- **NumPy data type** (`np.dtype`)
  - Only NumPy arrays (ndarray and subtypes) have a dtype
  - Dtype describes the data type of the elements of ndarrays

Python type vs. NumPy ndarray.dtype

Avoid mixing up **Python types** and **ndarray.dtype!**

E.g., `type("abc") → <class 'str'>`
`type(my_array) → <class 'numpy.ndarray'>`

- **Python type**
  - All Python objects have a type
  - The type of NumPy arrays is **`ndarray`**

- **NumPy data type** (`np.dtype`)
  - Only NumPy arrays (ndarray and subtypes) have a dtype
  - Dtype describes the data type of the elements of ndarrays

E.g., `my_array.dtype → dtype('int64')`
`type(my_array[0]) → <class 'numpy.int64'>`

NumPy - Array creation methods (1D)

```
a = np.array([0, 10, 20, 30, 40], dtype=np.int64)        # [ 0 10 20 30 40]
b = np.arange(5, dtype=np.int64)                         # [0 1 2 3 4]

c = np.zeros((5,), dtype=np.float32)                     # [0. 0. 0. 0. 0.]
d = np.ones((5,), dtype=np.bool_)                        # [ True  True  True  True  True]
e = np.full((5,), dtype=np.int32, fill_value=99)         # [99 99 99 99 99]

f = np.zeros_like(d)                                     # [False False False False False]
g = np.ones_like(e)                                      # [1 1 1 1 1]
h = np.full_like(c, fill_value=99)                       # [99. 99. 99. 99. 99.]

m = np.empty((5,), dtype=np.uint8)                       # [ 77 253   3   0   0]
n = np.empty_like(m, dtype=np.bool_)                     # [True  True  True  True False]
...
```

NumPy - Array creation methods (1D)

```
a = np.array([0, 10, 20, 30, 40], dtype=np.int64)        # [ 0 10 20 30 40]
b = np.arange(5, dtype=np.int64)                         # [0 1 2 3 4]

c = np.zeros((5,), dtype=np.float32)                     # [0. 0. 0. 0. 0.]
d = np.ones((5,), dtype=np.bool_)                        # [ True  True  True  True  True]
e = np.full((5,), dtype=np.int32, fill_value=99)         # [99 99 99 99 99]

f = np.zeros_like(d)                                     # [False False False False False]
g = np.ones_like(e)                                      # [1 1 1 1 1]
h = np.full_like(c, fill_value=99)                       # [99. 99. 99. 99. 99.]

m = np.empty((5,), dtype=np.uint8)                       # [ 77 253   3   0   0]
n = np.empty_like(m, dtype=np.bool_)                     # [True  True  True  True False]
```

...

No initialization of values. Undefined content.

NumPy - Examples for element-wise operations

Numeric → Numeric

```
-   (np.negative)
np.fabs
np.sign
np.round
np.floor
np.ceil
np.exp
np.log
np.sin
...
```

Numeric × Numeric → Numeric

```
+   (np.add)
-   (np.subtract)
*   (np.multiply)
/   (np.divide)
//  (np.floor_divide)
%   (np.mod)
**  (np.power)
np.minimum
np.maximum
...
```

NumPy - Examples for element-wise operations

Numeric × Numeric → Boolean

```
==    (np.equal)
!=    (np.not_equal)
<     (np.less)
<=    (np.less_equal)
>     (np.greater)
>=    (np.greater_equal)
np.isclose

...
```

Boolean × Boolean → Boolean

```
==    (np.equal)
!=    (np.not_equal)
&     (np.logical_and)
|     (np.logical_or)
np.logical_xor
...
```

Boolean → Boolean

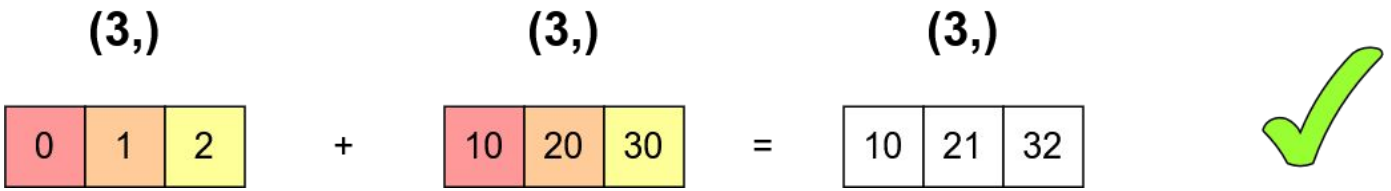
```
~     (np.logical_not)
```

NumPy - Element-wise operations

Element-wise operations can be applied on arrays with similar shape…

```
a = np.array([0, 1, 2], dtype=np.int64)          # a.shape == (3,)
b = np.array([10, 20, 30], dtype=np.int64)       # b.shape == (3,)

print((a+b).shape)                               # --> (3,)
print(a+b)                                       # --> [10 21 32]
```

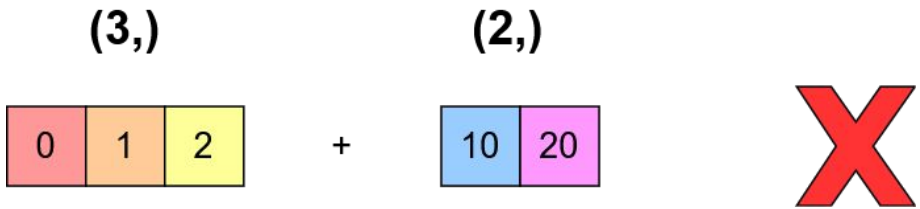


NumPy - Element-wise operations

Element-wise operations can be applied on arrays with similar shape…

```
a = np.array([0, 1, 2], dtype=np.int64)          # a.shape == (3,)
b = np.array([10, 20], dtype=np.int64)           # b.shape == (2,)

print(a+b)
# --> ValueError: operands could not be broadcast together with shapes (3,) (2,)
```

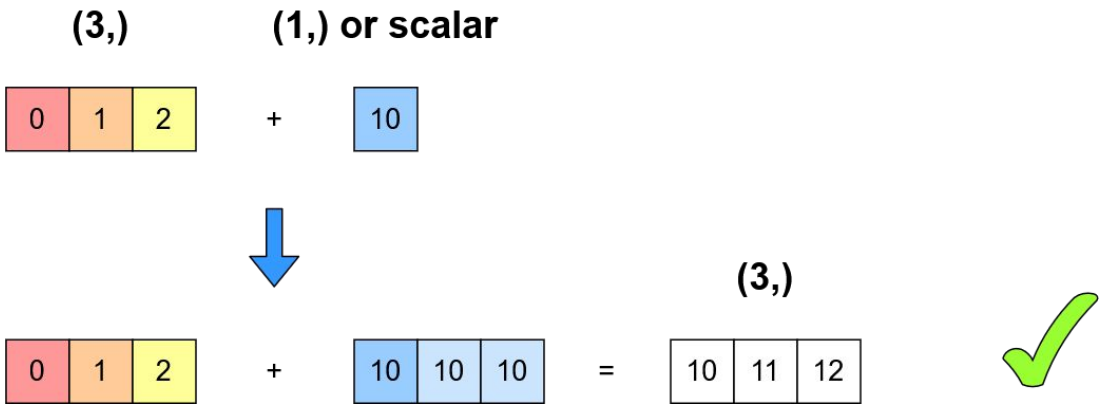


NumPy - Element-wise operations with broadcasting (1D)

… or **the shapes need to be compatible** in terms of **broadcasting.**

```
a = np.array([0, 1, 2], dtype=np.int64)      # a.shape == (3,)

print((a + 10).shape)                        # --> (3,)
print(a + 10)                                # --> [10 11 12]
```



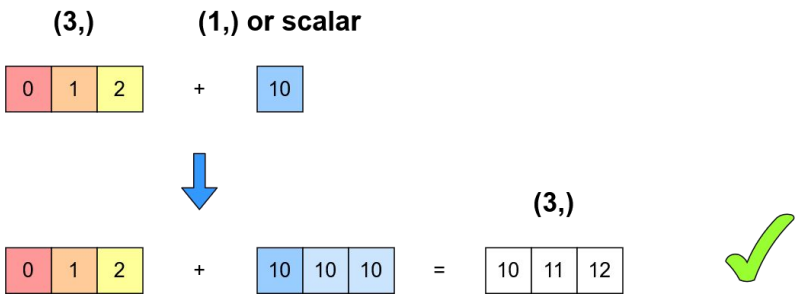
NumPy - Element-wise operations with broadcasting (1D)

Broadcasting is the **virtual repetition of elements** to match the shapes of two or more arrays.

```
a = np.array([0, 1, 2], dtype=np.int64)      # a.shape == (3,)

print((a + 10).shape)                        # --> (3,)
print(a + 10)                                # --> [10 11 12]
```

Broadcasting happens automatically in the context of **element-wise ops**!

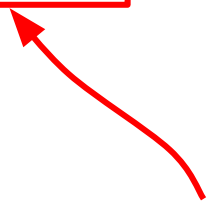


NumPy - Element-wise operations with broadcasting (1D)

Broadcasting is the **virtual repetition of elements** to match the shapes of two or more arrays.

```
a = np.array([0, 1, 2], dtype=np.int64)          # a.shape == (3,)
tens = np.full_like(a, fill_value=10)            # tens.shape == (3,)

print((a + tens).shape)                          # --> (3,)
print(a + tens)                                  # --> [10 11 12]
```



No need to physically create the repetitions.

Broadcasting is virtual and therefore more efficient than manual tiling / repetitions!

NumPy - Examples for collapse operations (1D)

[Numeric] → Numeric

```
np.sum      (np.ndarray.sum)
np.prod     (np.ndarray.prod)
np.amax     (np.ndarray.max)
np.amin     (np.ndarray.min)
np.mean     (np.ndarray.mean)
np.var      (np.ndarray.var)
np.std      (np.ndarray.std)
np.median
np.argmax   (np.ndarray.argmax)
np.argmin   (np.ndarray.argmin)
np.count_nonzero
...
```

[Boolean] → Boolean

```
np.any    (np.ndarray.any)
np.all    (np.ndarray.all)
np.count_nonzero
...
```

NumPy - Examples for collapse operations (1D)

[Numeric] → Numeric

```
np.sum      (np.ndarray.sum)
np.prod     (np.ndarray.prod)
np.amax     (np.ndarray.max)
np.amin     (np.ndarray.min)
np.mean     (np.ndarray.mean)
np.var      (np.ndarray.var)
np.std      (np.ndarray.std)
np.median
np.argmax   (np.ndarray.argmax)
np.argmin   (np.ndarray.argmin)
np.count_nonzero
...
```

[Boolean] → Boolean

```
np.any    (np.ndarray.any)
np.all    (np.ndarray.all)
np.count_nonzero
...
```

E.g., `np.sum(my_array)`

`my_array.sum()`

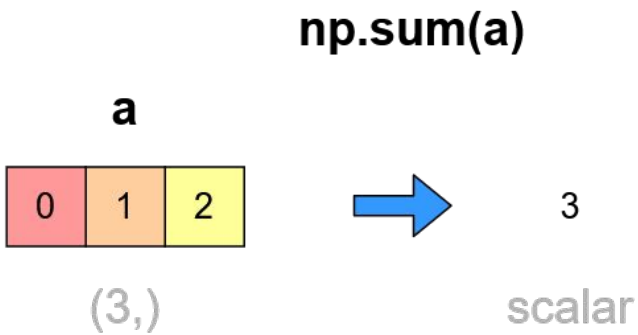
Identical results

NumPy - Collapse operations (1D)

(1D case) **Collapse operations** collapse an 1D array into a scalar.

```
a = np.array([0, 1, 2], dtype=np.int64)      # a.shape == (3,)

print(np.sum(a))                             # --> 3
```



Reminder: Python list indexing

Reminder: Python list indexing

```
my_list = [0, ["one"], True, "THREE", 4.0001]

print(my_list[0])          # 0
print(my_list[-2])         # THREE

print(my_list[1:3])        # [['one'], True]
print(my_list[1:])         # [['one'], True, 'THREE', 4.0001]
print(my_list[:-1])        # [0, ['one'], True, 'THREE']
print(my_list[4::-2])      # [4.0001, True, 0]
```

Individual integer index
Returns an element.

Reminder: Python list indexing

Reminder: Python list indexing

```
my_list = [0, ["one"], True, "THREE", 4.0001]
```

```
print(my_list[0])          # 0
print(my_list[-2])         # THREE
```

Individual integer index
Returns an element.

```
print(my_list[1:3])        # [['one'], True]
print(my_list[1:])         # [['one'], True, 'THREE', 4.0001]
print(my_list[:-1])        # [0, ['one'], True, 'THREE']
print(my_list[4::-2])      # [4.0001, True, 0]
```

Slicing
(Select elements from an interval with a regular step size)
Returns **a new list** with the selected elements.

Reminder: Python list indexing

Reminder: Python list indexing

```
my_list = [0, ["one"], True, "THREE", 4.0001]
```

```
print(my_list[0])          # 0
print(my_list[-2])         # THREE
```

Individual integer index
Returns an element.

```
print(my_list[1:3])        # [['one'], True]
print(my_list[1:])         # [['one'], True, 'THREE', 4.0001]
print(my_list[:-1])        # [0, ['one'], True, 'THREE']
print(my_list[4::-2])      # [4.0001, True, 0]
```

If you are not comfortable with slicing syntax, check it!

Slicing
(Select elements from an interval with a regular step size)
Returns **a new list** with the selected elements.

Four ways to index an 1D array!

NumPy - Array indexing (1D)

```
a = np.array([0, 10, 20, 30, 40], dtype=np.int64)        # a.shape == (5,)
print(type(a))                                           # <class 'numpy.ndarray'>
```

1

```
print(a[0])                                              # 0
print(a[-1])                                             # 40
print(type(a[-1]))                                       # <class 'numpy.int64'>
```

Individual integer index
Returns a scalar.

2

```
print(a[1:3])                                            # [10 20]
print(a[1:])                                             # [10 20 30 40]
print(a[:-1])                                            # [0 10 20 30]
print(a[4::-2])                                          # [40 20 0]
print(type(a[4::-2]))                                    # <class 'numpy.ndarray'>
```

Slicing
(Select elements from an interval with a regular step size)
Returns **a view** (still ndarray) **of the original array** with the selected elements.

NumPy - Array indexing (1D)

```
a = np.array([0, 10, 20, 30, 40], dtype=np.int64)        # a.shape == (5,)
print(type(a))                                           # <class 'numpy.ndarray'>
```

1

```
print(a[0])                                              # 0
print(a[-1])                                             # 40
print(type(a[-1]))                                       # <class 'numpy.int64'>
```

Individual integer index
Returns a scalar.

2

```
print(a[1:3])                                            # [10 20]
print(a[1:])                                             # [10 20 30 40]
print(a[:-1])                                            # [0 10 20 30]
print(a[4::-2])                                          # [40 20 0]
print(type(a[4::-2]))                                    # <class 'numpy.ndarray'>
```

Writing into the view will modify the original array too and vice versa!

Slicing
(Select elements from an interval with a regular step size)
Returns **a view** (still ndarray) **of the original array** with the selected elements.

NumPy - Array indexing (1D)

```
a = np.array([0, 10, 20, 30, 40], dtype=np.int64)          # a.shape == (5,)
```

3

```
print(a[[2, 3]])            # [20 30]
print(a[[]])                # []          ( a[[]].shape == (0,) )
print(a[[-2, 0, 1, 0]])     # [30 0 10 0]
print(a[[-1, -1]])          # [40 40]
print(type(a[[-1, -1]]))    # <class 'numpy.ndarray'>

my_idxs = np.array([2, 3], dtype=np.int64)
print(a[my_idxs])            # [20 30]
```

Indexing with a sequence of integers.
(Select multiple arbitrary elements by index)
Returns **a new array** with the selected elements.

NumPy - Array indexing (1D)

```
a = np.array([0, 10, 20, 30, 40], dtype=np.int64)          # a.shape == (5,)
```

3

```
print(a[[2, 3]])            # [20 30]
print(a[[]])                # []          ( a[[]].shape == (0,) )
print(a[[-2, 0, 1, 0]])     # [30 0 10 0]
print(a[[-1, -1]])          # [40 40]
print(type(a[[-1, -1]]))    # <class 'numpy.ndarray'>

my_idxs = np.array([2, 3], dtype=np.int64)
print(a[my_idxs])           # [20 30]
```

The sequence of integers can come from a list, or another array too (but not a tuple).

Indexing with a sequence of integers.
(Select multiple arbitrary elements by index)
Returns **a new array** with the selected elements.

NumPy - Array indexing (1D)

```
a = np.array([0, 10, 20, 30, 40], dtype=np.int64)          # a.shape == (5,)
```

4

```
print(a[[True, True, False, True, False]])          # [0 10 30]
print(a[[True, True, True, True, True]])            # [0 10 20 30 40]
print(a[[False, False, False, False, False]])       # []

my_mask = np.ones((5,), dtype=np.bool_)
print(my_mask)              # [ True  True  True  True  True]
print(my_mask.shape)        # (5,)
print(my_mask.dtype)        # bool
print(type(my_mask))        # <class 'numpy.ndarray'>
print(a[my_mask])           # [0 10 20 30 40]
print(a[my_mask].shape)     # (5,)
print(a[my_mask].dtype)     # int64
print(type(a[my_mask]))     # <class 'numpy.ndarray'>
```

Indexing with a boolean mask.

(Select multiple arbitrary elements by applying a mask)
Returns **a new array** with the selected elements.

NumPy - Array indexing (1D)

```
a = np.array([0, 10, 20, 30, 40], dtype=np.int64)          # a.shape == (5,)
```

4

```
print(a[[True, True, False, True, False]])          # [0 10 30]
print(a[[True, True, True, True, True]])            # [0 10 20 30 40]
print(a[[False, False, False, False, False]])       # []

my_mask = np.ones((5,), dtype=np.bool_)
print(my_mask)              # [ True  True  True  True  True]
print(my_mask.shape)        # (5,)
print(my_mask.dtype)        # bool
print(type(my_mask))        # <class 'numpy.ndarray'>
print(a[my_mask])           # [0 10 20 30 40]
print(a[my_mask].shape)     # (5,)
print(a[my_mask].dtype)     # int64
print(type(a[my_mask]))     # <class 'numpy.ndarray'>
```

The mask can come from a list of booleans, or another boolean array too.

Indexing with a boolean mask.

(Select multiple arbitrary elements by applying a mask)

Returns **a new array** with the selected elements.

NumPy - Array indexing (1D)

Indexing with a condition:
The mask can be created as a result of a boolean-valued function too…

```
a = np.array([0, 10, 20, 30, 40], dtype=np.int64)          # a.shape == (5,)
my_mask = a % 30 == 0
```

4

```
print(my_mask)              # [ True  False  False  True  False]
print(my_mask.shape)        # (5,)
print(my_mask.dtype)        # bool
print(a[my_mask])           # [0 30]
print(a[my_mask].shape)     # (2,)

print(a[a % 3 == 0])        # [0 30]
```

Indexing with a boolean mask.
(Select multiple arbitrary elements by applying a mask)
Returns **a new array** with the selected elements.

NumPy - Array indexing for **writing** (1D)

```
a = np.array([0, 10, 20, 30, 40], dtype=np.int64)   # a.shape == (5,)
```

1

```
a[-1] = 99
print(a)                    # [ 0 10 20 30 99]
```

Overwriting a single element by using an individual integer index

NumPy - Array indexing for **writing** (1D)

```
a = np.array([0, 10, 20, 30, 40], dtype=np.int64)   # a.shape == (5,)
```

1

```
a[-1] = 99.99
print(a)                    # [ 0 10 20 30 99]
```

Overwriting a single element by using an individual integer index

Must pay attention:

Right-handside element will be converted to match the datatype of the array.
E.g., If a float is written into an integer array, it will silently lose its fractional part…

NumPy - Array indexing for **writing** (1D)

```
a = np.array([0, 10, 20, 30, 40], dtype=np.int64)    # a.shape == (5,)
```

2

```
a[::2] = 42, 99, 11
print(a)                    # [42 10 99 30 11]
```

Overwriting multiple elements (in an interval with regular step size) **by slice indexing.**

NumPy - Array indexing for **writing** (1D)

```
a = np.array([0, 10, 20, 30, 40], dtype=np.int64)    # a.shape == (5,)
```

2

```
a[::2] = 42, 99, 11
print(a)                    # [42 10 99 30 11]
```

Selecting 3 indices ⟷ Overwriting these with 3 values.



Same number of elements on both sides of the assignment.

Overwriting multiple elements (in an interval with regular step size)
by slice indexing.

NumPy - Array indexing for **writing** (1D)

```
a = np.array([0, 10, 20, 30, 40], dtype=np.int64)    # a.shape == (5,)
```

2

```
a[::2] = 99
print(a)                    # [99 10 99 30 99]
```

Selecting 3 indices ⟷ Overwriting these with 1 value.

N vs. 1 elements on the two sides of the assignment → BROADCASTING
(Virtual repetitions)

Overwriting multiple elements (in an interval with regular step size)
by slice indexing.

NumPy - Array indexing for **writing** (1D)

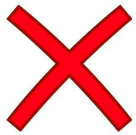
```
a = np.array([0, 10, 20, 30, 40], dtype=np.int64)    # a.shape == (5,)
```

2

```
a[::2] = 42, 99
# ValueError: could not broadcast input array from shape (2,) into shape (3,)
```

Selecting 3 indices ⟷ Overwriting these with 2 values.

N vs. M elements, N ≠ M ≠ 1 → BROADCASTING FAILS



Overwriting multiple elements (in an interval with regular step size)
by slice indexing.

NumPy - Array indexing for **writing** (1D)

```
a = np.array([0, 10, 20, 30, 40], dtype=np.int64)          # a.shape == (5,)
```

3

```
a[[1, -1, 0]] = 42, 99, 11
print(a)                  # [11 42 20 30 99]

a[[1, -1, 0]] = 99
print(a)                  # [99 99 20 30 99]
```

← **Broadcasting**

Overwriting multiple (arbitrary) **elements**
by providing a sequence of integer indices
(coming from a list or another array)**.**

NumPy - Array indexing for **writing** (1D)

```
a = np.array([0, 10, 20, 30, 40], dtype=np.int64)          # a.shape == (5,)
my_idxs = np.array([1, -1, 0], dtype=np.int64)
my_new_values = np.array([42, 99, 11], dtype=np.int64)
```

3

```
a[my_idxs] = my_new_values
print(a)                    # [11 42 20 30 99]
```

Overwriting multiple (arbitrary) **elements by providing a sequence of integer indices** (coming from a list or another array)**.**

Of course, both the indices and the values to use for overwriting **can come from other arrays…**

NumPy - Array indexing for **writing** (1D)

```
a = np.array([0, 10, 20, 30, 40], dtype=np.int64)          # a.shape == (5,)
my_mask = a % 30 == 0
print(my_mask.shape)                                       # (5,)
print(my_mask)                                             # [ True False False  True False]
```

4

```
a[my_mask] = 42, 77
print(a)                    # [42 10 20 77 40]
```

Overwriting multiple elements selected by a boolean mask.

NumPy - Array indexing for **writing** (1D)

```
a = np.array([0, 10, 20, 30, 40], dtype=np.int64)          # a.shape == (5,)
```

4

```
a[a % 30 == 0] = 99
print(a)                    # [99 10 20 99 40]
```

← **Broadcasting**

Overwriting multiple elements selected by a boolean mask.

The mask (boolean array) **can be created just for this purpose** without assigning it to a variable…

NumPy - Finding index of **all** elements with a condition

```
a = np.array([0, 10, 20, 30, 40], dtype=np.int64)          # a.shape == (5,)
my_mask = a % 30 == 0

print(my_mask)              # [ True  False  False  True  False]
print(my_mask.shape)        # (5,)
print(my_mask.dtype)        # bool
```

```
my_idxs_tuple = np.where(my_mask)
print(my_idxs_tuple)                    # (array([0, 3]),)
print(type(my_idxs_tuple))              # <class 'tuple'>

my_idxs_arr, = my_idxs_tuple            # or:  my_idxs_arr = my_idxs_tuple[0]
print(my_idxs_arr)                      # [0 3]
```

`np.where`: Converting a mask to integer indices

A use case: Find the index of all elements with a condition / mask!

NumPy - Finding index of the **first** element with a condition

```
a = np.array([0, 10, 20, 30, 40], dtype=np.int64)          # a.shape == (5,)
my_mask = a > 15

print(my_mask)              # [ False  False  True  True  True]
print(my_mask.shape)        # (5,)
print(my_mask.dtype)        # bool

first_idx = np.argmax(my_mask)
print(first_idx)                  # 2
print(my_mask[first_idx])         # True

print(a[first_idx])               # 20
```

`np.argmax`: Finding the index of the largest element

A use case: Find the index of the first element with a condition / mask!

NumPy - Finding index of the **first** element with a condition

```
a = np.array([0, 10, 20, 30, 40], dtype=np.int64)          # a.shape == (5,)
my_mask = a > 15

print(my_mask)              # [ False  False  True  True  True]
print(my_mask.shape)        # (5,)
print(my_mask.dtype)        # bool
```

np.argmax on a mask will find the index of the first True value, as True > False (if there are any True values).

```
first_idx = np.argmax(my_mask)
print(first_idx)                    # 2
print(my_mask[first_idx])           # True

print(a[first_idx])                 # 20
```

`np.argmax`: Finding the index of the largest element

A use case: Find the index of the first element with a condition / mask!

NumPy - Finding index of the **first** element with a condition

```
a = np.array([0, 10, 20, 30, 40], dtype=np.int64)          # a.shape == (5,)
my_mask = a > 15

print(my_mask)              # [ False  False  True  True  True]
print(my_mask.shape)        # (5,)
print(my_mask.dtype)        # bool

first_idx = np.argmax(my_mask)
print(first_idx)                    # 2
print(my_mask[first_idx])           # True

print(a[first_idx])                 # 20
```

Binary repr: 1 > 0

np.argmax on a mask will find the index of the first True value, as True > False
(if there are any True values).

Need to check if the mask is True in this index
(For an all False mask, argmax will return the index of the first False)

`np.argmax`: Finding the index of the largest element

A use case: Find the index of the first element with a condition / mask!

NumPy - Finding index of the **first** element with a condition

```
a = np.array([0, 10, 20, 30, 40], dtype=np.int64)          # a.shape == (5,)
my_mask = a > 15

print(my_mask)              # [ False  False  True  True  True]
print(my_mask.shape)        # (5,)
print(my_mask.dtype)        # bool

first_idx = np.argmax(my_mask)
print(first_idx)            # 2
print(my_mask[first_idx])   # True
print(first_idx)            # 2
```

If we only need to find the **first element** with a condition / mask,
np.argmax is more efficient.
Short-circuit behavior: It stops on finding the datatype maximum.
For boolean arrays this is the first True value in the array.

`np.argmax`: Finding the index of the largest element

A use case: Find the index of the first element with a condition / mask!